

SLEEPY-HOLLOW-ARCHITEKTUR, TEIL 2

Ich packe meinen Koffer ...

Von den Anforderungen zu einer testbaren, modularen, wandelbaren Implementation:
Wie geht das?

Dieser Artikel möchte Ihnen zeigen, wie Sie über eine Anforderungsanalyse mit Slicing (siehe die Serie von [1] bis [2]) und das Sleepy-Hollow-Architekturmuster [3] einem sehr geraden Weg von den Anforderungen zu einer testbaren, modularen, wandelbaren Implementation folgen können.

Lassen Sie uns gleich in die Praxis einsteigen. Das Handwerkszeug habe ich in den bisherigen Artikeln ausführlich vorgestellt. Jetzt soll es endlich in einem etwas größeren Beispiel zum Einsatz kommen.

Das Szenario

Es soll um ein Softwaresystem gehen, mit dem ein Team gemeinsam Ordnungen herstellen kann. Jedes Teammitglied soll die Positionen in einer Ordnung allerdings nicht absolut bewerten, sondern relativ.

- **Beispiel Priorisierung:** Aufgaben sind in eine Ordnung in Bezug auf die zeitliche Abfolge ihrer Erledigung zu bringen. In welcher Reihenfolge sollen die Aufgaben A1, A2, A3 bearbeitet werden? Teammitglied Paul findet, die Reihenfolge sollte A2, A3, A1 sein; Teammitglied Maria hingegen denkt, die Prioritäten sollten A3, A2, A1 sein; und Peter schließlich hält auch A2, A3, A1 für die beste Reihenfolge.
- **Beispiel Komplexitätsschätzung:** Issues sind in eine Ordnung in Bezug auf den Aufwand, der für ihre Umsetzung nötig ist, zu bringen. Welches der Issues I1, I2, I3, I4 ist komplexer als welches andere und wird deshalb wahrscheinlich mehr Zeit brauchen? Peter meint, das komplexeste Issue sei I3, dem I1, I4, I2 in absteigender Komplexität folgen; Paul hingegen sieht es so: I1, I3, I2, I4; und Maria schätzt die Komplexität so: I3, I4, I1, I2.

In Ermangelung von Kriterien, aus denen sich absolute Werte für Priorität oder Komplexität ableiten ließen, ist eine solche subjektive, relative Schätzung ehrlicher und einfacher als eine, die künstlich absolute Werte erzwingt (zum Beispiel Story Points). Teammitglieder müssen nur „ein Gefühl haben“, dass eine Position im Hinblick auf das jeweilige Ordnungsthema vor einer anderen stehen sollte.

Eine gemeinschaftliche resultierende Ordnung des Teams kann aus den verschiedenen individuellen Ordnungen der Teammitglieder leicht berechnet werden. Für jede Position werden die Indizes der Stellen

summiert, an denen sie steht. Je geringer die Summe, desto weiter vorne steht die Position in der Ordnung des gesamten Teams.

Bild 1 zeigt das für die beiden Beispiele. Zur Erläuterung die Bestimmung der Prioritätsordnung:

- Aufgabe A1 steht bei Peter und allen anderen an dritter Stelle. Sie hat immer den Index 3, die Summe ihrer Indizes ist also $3+3+3=9$. Aufgabe A2 steht bei Peter und Paul an erster Stelle (Index 1) und bei Maria an zweiter Stelle (Index 2); daraus ergibt sich die Summe 4. A3 steht bei Peter und Paul an zweiter Stelle (Index 2) und bei Maria an erster Stelle (Index 1); die Summe ist 5.
- Die Positionen mit ihren Summen: (A1,9), (A2,4), (A3,5).
- Sortiert nach aufsteigenden Summen ergibt sich als kalkulierte Ordnung für das Team: A2, A3, A1. Diese Ordnung ist sozusagen das Ergebnis der „wisdom of the crowd“.

Bei diesem Vorgehen wird (zunächst) bewusst nicht berücksichtigt, dass zum Beispiel der Abstand zwischen A1 und A2 viel größer ist ($9-4=5$) als der zwischen A3 und A2 ($5-4=1$). Es geht um eine schnelle, grobe Einschätzung der Ordnung „aus dem Bauch heraus“.

Das Softwaresystem soll eine Möglichkeit bieten, Positionen zu sammeln, die dann von Teammitgliedern in individuelle Ordnungen gebracht werden. Diese individuellen Ordnungen werden zu einer gemeinschaftlichen Ordnung verrechnet und angezeigt.

Teammitglieder sollen für die Bestimmung einer gemeinschaftlichen Ordnung nicht co-located sein müssen. Sie sollen auch nicht zeitlich beschränkt sein; die gemeinschaftliche Ordnung soll also asynchron stattfinden können.

	A	B	C	D	E	F	G	H	I	J
1	Priorisierung									
2		Peter	Paul	Maria			A1	A2	A3	
3	1	A2	A2	A3		Peter	3	1	2	
4	2	A3	A3	A2		Paul	3	1	2	
5	3	A1	A1	A1		Maria	3	2	1	
6							9	4	5	
7						Ergebnis	A2, A3, A1			
8										
9	Komplexitätsschätzung									
10		Peter	Paul	Maria			I1	I2	I3	I4
11	1	I3	I1	I3		Peter	2	4	1	3
12	2	I1	I3	I4		Paul	1	3	2	4
13	3	I4	I2	I1		Maria	3	4	1	2
14	4	I2	I4	I2			6	11	4	9
15						Ergebnis	I3, I1, I4, I2			
16										

Berechnung der Ordnung für das Team (Bild 1)

Erste Einschätzung

Wie ist Ihre Einschätzung dieses Projekts nach dieser Anforderungspräsentation? Das ist natürlich nur eine Skizze, und Sie stehen in keinem Dialog mit einem Auftraggeber. Das ist für Sie in dieser Größenordnung vielleicht ungewohnt. Aber nehmen Sie es als Übung und lassen Sie sich darauf ein, selbst wenn ich im Folgenden andere Annahmen machen sollte als Sie. Es werden auch Lücken bleiben, die Sie gern schließen würden, ich aber offen lasse. Das ist dem Monolog geschuldet, den dieser Artikel darstellt, und auch seinem notwendig begrenzten Umfang.

Es geht allerdings nicht wirklich darum, diese Idee marktreif umzusetzen (so sehr mir das immer mal wieder in den Fingern juckt), sondern um die Demonstration der Anwendung von Slicing und Sleepy Hollow. Das kann auch mit divergierenden Annahmen und Lücken geschehen.

Damit im Hinterkopf: Wie ist dieses Projekt einzuschätzen? Machbar? Wie ist der Aufwand? Was braucht es dafür? Wie würden Sie vorgehen? Was ist Ihr Reflex?

So viel kann ich sagen: Das ist gut machbar. Der Aufwand ist überschaubar. Ich habe es auch schon mal umgesetzt [4], die Implementation ist allerdings im Web nicht mehr lauffähig. Das hat Spaß gemacht; ich fand das Tool sehr nützlich und „mal etwas anderes“ als das ewige Planning Poker.

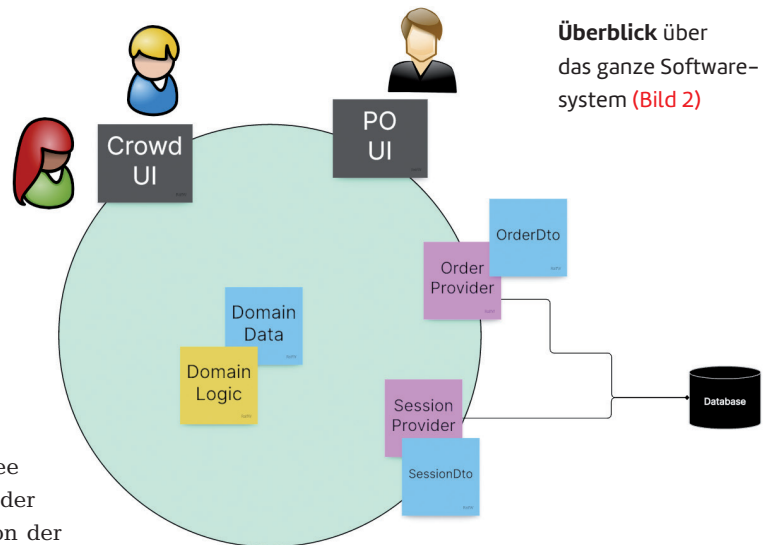
Was braucht es dafür?

- Entscheidungen für Technologien: Frontend, Backend, Persistenz und so weiter.
- Das ist sicher ein verteiltes System, da die Teammitglieder nicht co-located sein müssen.
- Die Usability bei der individuellen Ordnung sollte dem relativen, vergleichenden Schätzen gerecht werden. Drag-and-drop kommt Ihnen da vielleicht in den Sinn, um die Positionen leicht in eine Wunschordnung zu bringen.
- Wie steht es mit der Security? Wie viel Aufwand muss dafür getrieben werden? Meine Meinung: Es gibt kein Security-Problem. Die Daten, um die es geht, sind sehr flüchtig, nicht persönlich und enthalten keine Geheimnisse.
- Gibt es ein Skalierungsproblem? Meine Meinung: Nein, denn die Datenmengen sind marginal, die Zugriffsraten sind minimal, und die entscheidende Berechnung ist trivial und kann auch noch auf einem Client stattfinden.
- Die Domäne scheint einfach. Im Wesentlichen besteht sie aus der Logik, die individuelle Ordnungen der Teammitglieder zu einer gemeinsamen verrechnet.

Ist das eine Aufgabe, die etwas Systematik trägt, weil ich sie nicht „einfach runtercodieren“ kann? Aber sicher! Also los ...

Das ganze Softwaresystem aus 10 000 Meter Flughöhe

Die formellere Anforderungsanalyse mit Slicing beginnt in großer Flughöhe mit einem Blick auf das gesamte Softwaresystem. Was ist von dort aus mindestens zu sehen? Bild 2 zeigt den Überblick:

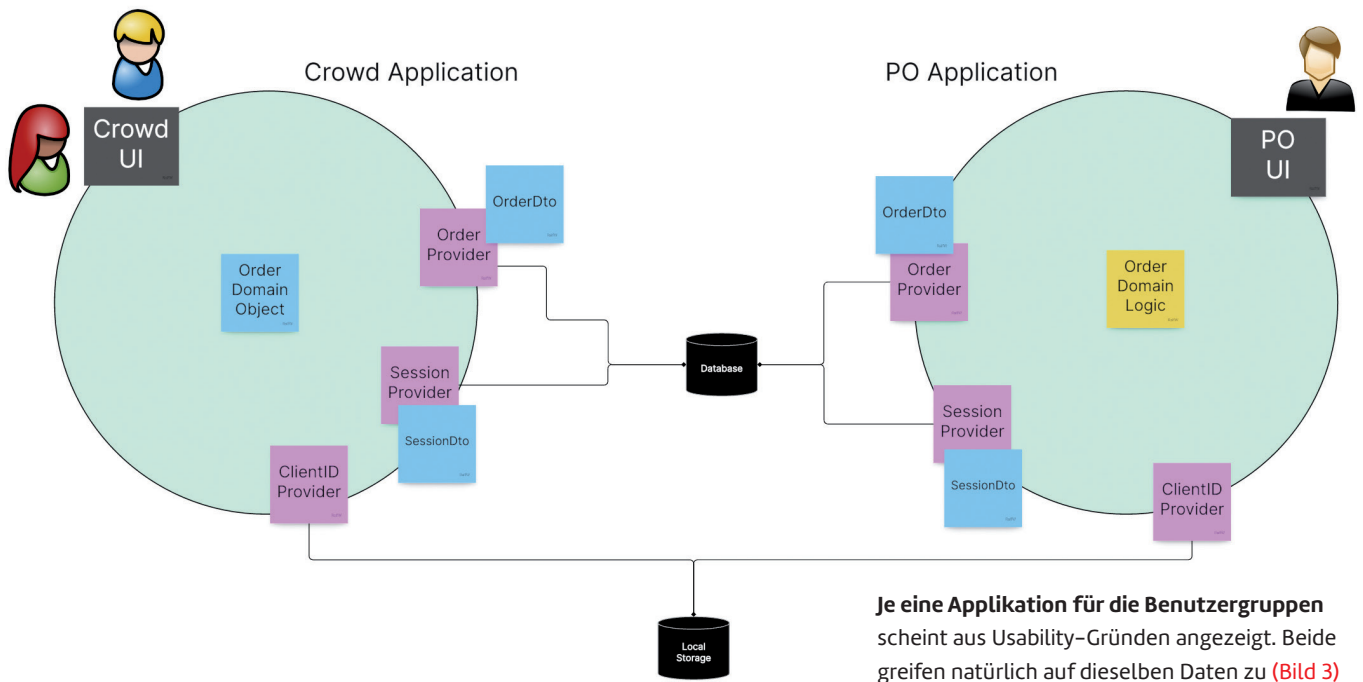


Überblick über das ganze Softwaresystem (Bild 2)

- Ich identifiziere zwei Benutzergruppen (Rollen): Da ist der PO, der die zu ordnenden Positionen festlegt; er leitet die „Ordnungssitzung“. Und da sind Teammitglieder (Crowd), die die Positionen in individuelle Ordnungen bringen. Jede Benutzergruppe verdient eine eigene Benutzerschnittstelle. Die Benutzerschnittstellen bilden den Rahmen für Interaktionen von Benutzern mit dem Softwaresystem.
- Andererseits braucht das Softwaresystem eine Ressource, eine Datenbank, um die Sitzungen mit ihren Positionen (oder Einträgen) beziehungsweise den verschiedenen Ordnungen der Teammitglieder zu speichern. Dafür scheinen mir zwei getrennte Provider nötig, die die persistenten Daten in interne Datenstrukturen transformieren.
- Außerdem gibt es noch nicht näher bestimmte Domänenlogik, die mit den Daten arbeitet, und vielleicht auch noch ausgewiesene Domänendatenstrukturen auf einem höheren Abstraktionsniveau als die DTOs der Provider.

Das sind alles keine tiefen Einsichten. Mich überrascht hier nichts. Das zu erkennen hat mich fünf Minuten gekostet. Dennoch ist dieser Überblick nützlich, denn er bietet mir eine erste Checkliste für Entscheidungen:

- Welche Technologie soll für die Benutzerschnittstellen zum Einsatz kommen? Ich entscheide mich zunächst der Einfachheit halber für ein Console-UI. Einem Web-UI steht später nichts im Wege, doch zu Anfang würde es mir mehr Kopfschmerzen in puncto Gestaltung und Implementation bereiten. Beides ist für das, worum es mir im Artikel geht, jedoch unwesentlich. Welche UIs es gibt, ist wichtig, nicht aber, wie genau sie ausgeprägt sind.
- Welche Technologie soll für die Datenbank zum Einsatz kommen? Auch wenn ich mir die Interaktionen noch nicht näher angeschaut habe, scheint mir nahezuliegen, dass die Zugriffsmuster sehr, sehr simpel sind. Im Grunde dreht sich alles um ein „Dokument“: eine Sitzung (Session) mit zu ordnenden Einträgen (Items). Zu einer Sitzung gehören verschiedene Ordnungen der Teammitglieder. Alles könnte in einem JSON-Objekt gespeichert werden. Aber irgendwie fühlt es sich nicht so gut an, mehrere Teammitglieder ►



potenziell gleichzeitig ein Sitzungsdocument mit ihren Ordnungen verändern zu lassen. Deshalb sehe ich die individuellen Ordnungen getrennt von den Sitzungen. Und wie sollen die Daten persistiert werden? Ich würde mal ganz einfach anfangen mit ... einem Dateisystem. Wenn ich die Daten in meiner Dropbox ablege, können mehrere Prozesse sehr einfach darauf zugreifen. Die Details sind in den Providern gekapselt. Wenn ich später meine, eine MongoDB einsetzen zu müssen, implementiere ich nur die Provider neu. Auch das Persistenzmedium ist für das, worum es mir hier geht, unwesentlich. Dass es eines gibt und in welcher Granularität es angesprochen wird, ist entscheidend; das drückt sich in den Providern aus.

- Soll das Softwaresystem verteilt laufen? Sicher. Das ist nötig, um den Teammitgliedern Freiheit in Bezug auf Ort und Zeit für ihre Ordnungen zu geben.

Näheres zu Domäne oder Datenstrukturen will ich an dieser Stelle noch nicht entscheiden, dazu möchte ich näher heranzoomen ...

Applikationen für jede Rolle

Die Verteilung des Codes für das Softwaresystem wird von nichtfunktionalen Anforderungen getrieben. Wenn Logik eine nichtfunktionale Anforderung nicht erfüllen kann, dann muss verteilt werden. Das ist offensichtlich hier der Fall: Die Rollen haben sehr unterschiedliche Anforderungen an die Benutzerschnittstelle. PO und Crowd gehen mit den Sitzungen und ihren Einträgen jeweils ganz anders um. Das scheint mir konsequent ausgedrückt durch spezialisierte Applikationen für die Rollen (Bild 3).

Bei dieser Spaltung des Softwaresystems frage ich mich, wie Benutzer sich ihren Applikationen gegenüber identifizieren (Authentifizierung). Ist eine Identifikation überhaupt nötig? Ja, ich denke, eine Identifikation ist aus zwei Gründen nötig:

- Ein PO soll nur seine Sitzungen bearbeiten können. Sitzungen haben zwar eine ID, doch wenn die außerhalb des Teams bekannt würde, könnte ein anderer PO die Sitzung einsehen. Das ist zwar kein großes Problem, glaube ich, aber das zu unterbinden ist einfach mit einer Identifikation des PO.
- Ein Teammitglied soll den Einträgen einer Sitzung nicht mehrere Ordnungen geben können. Veränderung ist zwar möglich, doch dadurch wird keine weitere Ordnung angelegt. Also muss erkannt werden, wer gerade die Einträge einer Sitzung „in Ordnung bringen will“.

Damit bei der Identifikation keine persönlichen Daten notwendig werden, deren Validität und Zugehörigkeit auch noch geprüft werden müssten, will ich es mir einfach machen: Jeder Client, den PO oder Teammitglieder nutzen, generiert eine ID bei der ersten Nutzung und merkt sie sich. Fertig. Dass innerhalb einer Sitzung die Clients gewechselt werden, ist unwahrscheinlich. Dass das (bei Teammitgliedern) „zum Mogeln“ bewusst gemacht wird, ist möglich, aber ich will es nicht mit mehr Aufwand berücksichtigen.

Also reicht es, bei den Applikationen noch einen weiteren Provider für die Client-ID hinzuzufügen, die in irgendeiner Form von lokalem Speicher (auf der Festplatte oder im Browser) gehalten wird.

Die Domäne habe ich nach ein wenig Nachdenken jetzt ebenfalls differenziert:

- In der PO-Applikation ist Domänenlogik nötig, um all die individuellen Ordnungen der Teammitglieder zu einer gemeinsamen zu verrechnen. Dort gibt es zu Recht den Überblick über alle eingereichten Ordnungen.
- In der Crowd-Applikation für die Teammitglieder ist wahrscheinlich nur eine Datenstruktur nötig, die es einfach macht, Positionen einer Sitzung neu anzuordnen. Logik für „Berechnungen“ ist dort nicht gefragt.

So weit der erste Zoomschritt in das Softwaresystem hinein. Sind damit alle nichtfunktionalen Anforderungen bedacht? Nein. Also noch mal hineinzoomen ...

Worker für die Sicherheit

Die Persistenz findet auf einem anderen Rechner statt als den Client-Rechnern für PO und Teammitglieder. Von Verteilung würde ich deshalb aber noch nicht sprechen. Wenn ein PO-Client auf eine Festplatte auf einem Server zugreift, tut er das mit eigenem Code. Das Persistenzmedium erscheint ihm als „dumm“; in der Nutzung der APIs des Persistenzmediums steckt die Intelligenz, die liegt beim Client.

Dafür braucht ein Client jedoch das Wissen über die APIs und auch noch unmittelbare Zugriffsrechte auf das Persistenzmedium. Bei einer Inhouse-Desktop-Applikation würde ich darüber nicht weiter nachdenken. Wenn die Applikationen jedoch potenziell im Web laufen und dann auch noch Code im Browser am Start ist ... würde ich die APIs und Credentials nicht in den Clients haben wollen.

Diese minimale Sicherheitsanforderung veranlasst mich dazu, ein Backend für die Datenbank auszuprägen. Das Softwaresystem zerfällt damit in drei Worker (Bild 4). Wie die Worker zu ihren Zugriffsrechten auf den Datenbankserver kommen (Autorisierung), ist eine andere Sache. Die lasse ich hier aus, weil ich schon mit der bisherigen Sicherheitsanforderung zu einem Backend-Worker gekommen bin. Mehr wollte ich nicht.

Im Worker für die Datenbank erwarte ich keine Domänenlogik. Eine solche könnte dort zentral angesiedelt werden, falls sie von der Nähe zu den Daten profitieren würde, doch in diesem Szenario ist das nicht der Fall. Das Backend ist insofern im Grunde nur ein Gateway zu den konkreten Funktionen für den Umgang mit dem Persistenzmedium.

Die Frontends sind Clients, die in Terminalfenstern laufen. Für meine Zwecke hier ist das der kleinstmögliche Einstieg.

Aber wie setze ich das Backend um? Es soll HTTP-Endpunkte für den Aufruf der Frontends bieten und auf meine Dropbox zugreifen. Ich könnte es mit C# oder Typescript unter Nutzung des Dropbox-API realisieren. Da ich jedoch Fan der Automatisierungsplattform Make.com bin, sehe ich natürlich überall Nägel für diesen Hammer: Das Backend realisiere ich in Form von Make.com-Szenarien. In Make.com ist der Zugriff auf die Dropbox trivial und schön visuell.

In Bild 4 unterscheidet sich das Backend in der grundsätzlichen Darstellung nicht von den Frontends. Das ist korrekt: Egal wie die Implementation aussieht, es wird Portale, Provider und Domäne geben. Die werden in immer gleicher Weise dargestellt. Die Ausprägungen hingegen mögen stark variieren:

- Wenn ich mich entscheiden sollte, das Crowd-Frontend mit C# und das PO-Frontend mit Python zu implementieren, ist der Code für die gleich benannten Provider natürlich unterschiedlich.
- Ein Provider in C# sieht sicher anders aus als einer, der in Make.com den Zugriff auf die Dropbox tatsächlich durchführt.

Die „Kärtchen“ in den Abbildungen sind nur Platzhalter. Sie zeigen an, was ich mindestens brauche und nicht vergessen darf. Implementationsdetails sind auf dieser Flughöhe nicht zu sehen.

Bisher hat die Flughöhe auch immer noch einen Überblick über das Gesamtsystem gestattet. Ich bin breadth-first vorgegangen. Mit dem nächsten Schritt näher ans System heran wird sich das ändern. Ich muss mich konzentrieren, um genauer hinschauen zu können.

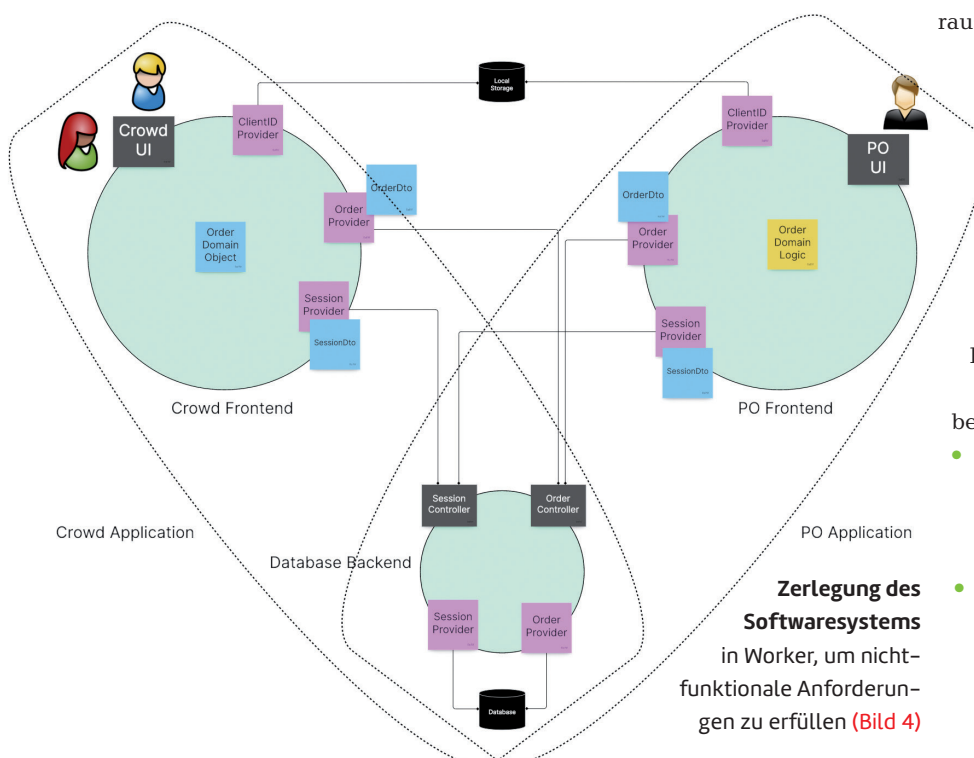
Weiter geht es mit dem konkreten Umgang der Rollen mit den Frontends. Wie soll die Bedienung aussehen? Daraus leiten sich Schnittstellen und Datenstrukturen ab.

Das Frontend für den PO

Der PO stößt die Ordnung von Positionen für einen gewissen Zweck an. Er definiert eine Sitzung mit ihren Positionen, die anschließend von den Teammitgliedern individuell geordnet werden sollen – um am Ende zu einer gemeinschaftlichen Ordnung verrechnet zu werden.

Ich sehe ganz grob folgende Aktivitäten beim PO:

- Der PO legt eine Sitzung mit einer kurzen Beschreibung an, zum Beispiel „Sprint KW 19: Ordne die User Stories in absteigender Komplexität an.“
- Der PO füllt die Sitzung mit Einträgen, das heißt den zu ordnenden Positionen, zum Beispiel „User können sich mit E-Mail und Passwort anmelden“, „User können ihr Passwort zurücksetzen“, „User können ►



Zerlegung des Softwaresystems
in Worker, um nicht-funktionale Anforderungen zu erfüllen (Bild 4)

sich mit E-Mail und Passwort registrieren“, „User bekommen eine Registrierungsbenachrichtigung per E-Mail und können sie bestätigen“.

- Der PO veröffentlicht die Sitzung zur Ordnung durch die Teammitglieder.
- Der PO verfolgt die Entwicklung der gemeinschaftlichen Ordnung.

Das ist das Mindeste. Weitere Aktivitäten könnten sein: Der PO schaut sich ältere Sitzungen an, der PO löscht Sitzungen, der PO verändert eine Sitzung, der PO leitet eine neue Sitzung von einer vorherigen ab, Veränderungen an der gemeinschaftlichen Ordnung werden automatisch dargestellt ...

Im Slicing können Sie den Umgang mit einem Softwaresystem in drei Schritten konkretisieren:

- Perspektiven
- Dialoge
- Interaktionen

Wie bei Applikationen und Workern ist kein Slice zwingend zu schneiden. Sie greifen dort zu, wo es nützlich für Sie ist, um Klarheit zu gewinnen. Ziel sind die Interaktionen. Sie repräsentieren testbare Eintrittspunkte im Code.

Die Frage ist: Was soll automatisiert testbar werden, und was soll andererseits ein Detail der Benutzerschnittstelle bleiben und nicht automatisiert getestet werden?

Aus meiner obigen Beschreibung der PO-Aktivitäten leite ich schon mal zwei testwürdige Interaktionen ab:

- Veröffentlichung einer Sitzung: Die Sitzung mit ihren Einträgen wird im Backend gespeichert. Anschließend können die Teammitglieder mit ihrem Crowd-Frontend darauf zugreifen.
- Anzeige des aktuellen Stands der gemeinschaftlichen Ordnung für die Reflexion mit dem Team: Die von den Teammitgliedern eingereichten individuellen Ordnungen werden vom Backend geladen und zu einer gemeinschaftlichen Ordnung verrechnet.

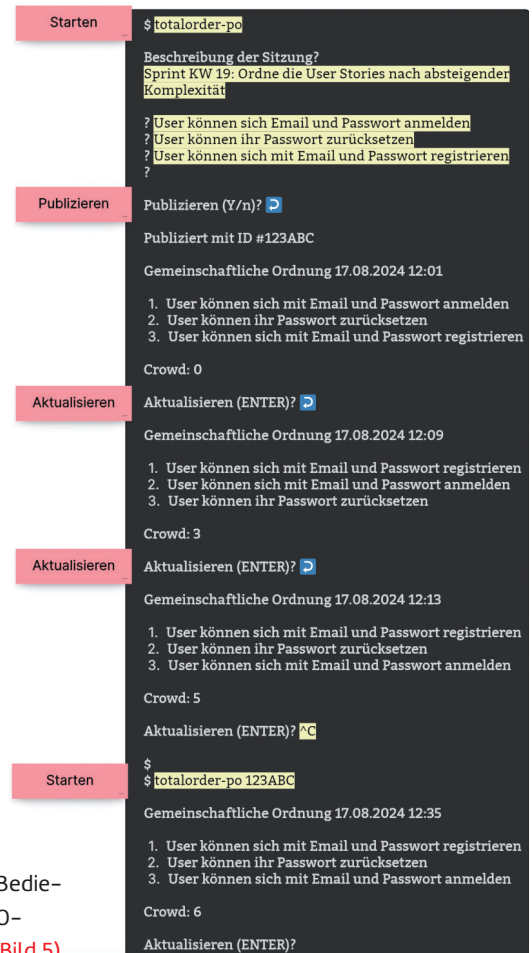
Wie wird eine Sitzung zusammengestellt? Das ist ein Detail der Benutzerschnittstelle. Im allereinfachsten Fall könnte ein PO eine JSON-Struktur in ein Textfeld eintragen. Aber das ist natürlich nicht benutzerfreundlich, also wird das UI es dem PO etwas bequemer machen, die Daten zu sammeln.

Wenn das ebenfalls testwürdig sein sollte, kann ich weitere Interaktionen explizieren, zum Beispiel:

- Sitzung anlegen mit einer kurzen Beschreibung.
- Sitzung um einen Eintrag erweitern.
- Einen Eintrag aus einer Sitzung löschen.

Der Einfachheit halber möchte ich mich jedoch auf die beiden obigen Interaktionen konzentrieren: Veröffentlichung, Reflexion.

Jetzt bin ich aber schon gleich auf der Ebene der Interaktionen gelandet. Was ist mit Perspektiven und Dialogen? Wie gesagt: Sie sind nicht zwingend für das Ziel von Slicing. In diesem Fall brauche ich sie nicht, glaube ich. Die Benutzerschnittstelle ist nicht weitläufig, also sind Perspektiven über-



Skizze der Bedienung des PO-Frontends (Bild 5)

flüssig. Und ich denke, ich komme mit zwei Dialogen aus:

- Publizieren
- Reflektieren

Oder beide Interaktionen könnten sogar in einem Dialog zusammengefasst sein. Bild 5 skizziert, wie ich mir den Umgang mit dem PO-Frontend in einem Terminalfenster vorstelle. Hervorgehoben habe ich Benutzereingaben und Interaktionen. Hinzugekommen ist noch eine Interaktion fürs Starten, um auch bei erneutem Programmaufruf auf eine vorherige Sitzung zugreifen zu können.

Aber ich könnte natürlich auch eine stärker visuell ausgerichtete Benutzerschnittstelle entwerfen (Bild 6). An den Interaktionen ändert das nichts. Auch hinter einem GUI oder einem Web-UI stehen dieselben Entry-Point-Funktionen.

Sie sehen, aus Sicht des Entwicklers ist das UI zunächst nicht so wichtig. Es trägt nichts zur „eigentlichen Verarbeitung“ bei. Das Verhalten der Software insgesamt, das heißt die Transformation von Input in Output, findet hinter dieser Fassade statt.

Das macht die Fassade nicht unwichtig. Ohne gute Usability wird auch die schönste Transformation nicht genutzt. Doch ohne saubere Transformation, das heißt korrekte, verständliche, wandelbare Softwarestrukturen, lohnt sich kein Aufwand für Usability. Deshalb ist das UI im Detail für mich zweitrangig. Ich will mir (zunächst) nur so viel davon klarmachen, wie

ich brauche, um die Interaktion zu erkennen, die mir die Slicing-Entry-Points auf den Sleepy-Hollow-Processor-Schnittstellen beschreiben.

Wenn ich sie nach Bild 2 schon hätte listen können, hätte ich mir einigen Aufwand sparen können. Doch auf dieser Flughöhe habe ich sie noch nicht gesehen. Also habe ich hineingezoomt. Die Slicing-„Zoomstufen“-Perspektive konnte ich allerdings überspringen. Und bei Interaktionen in Dialogen bin ich nicht streng „von oben nach unten“ vorgegangen, weil der Scope des Frontends so überschaubar ist.

Das Frontend für die Crowd

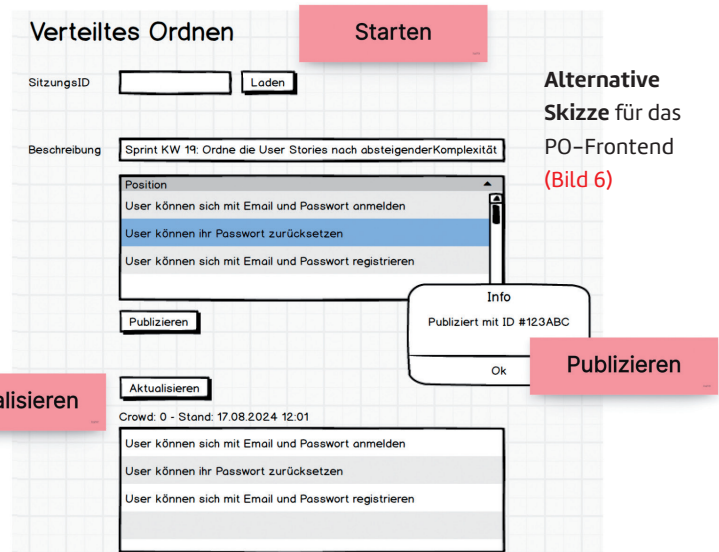
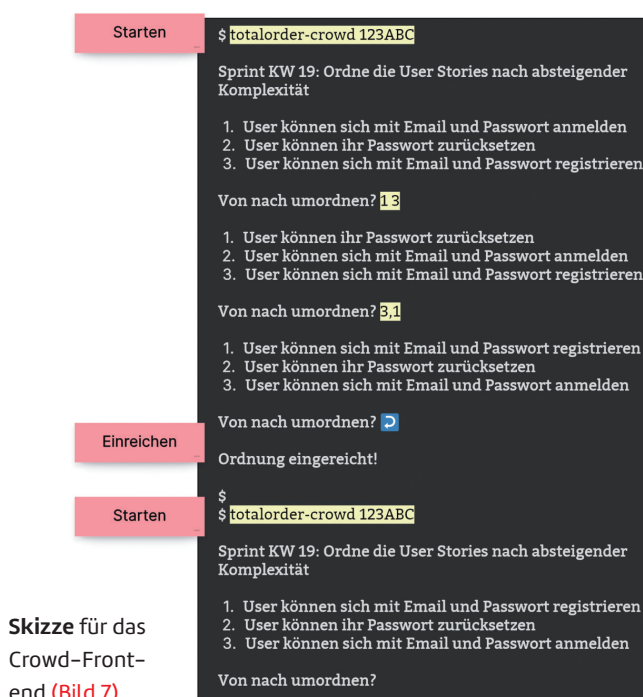
Für die Teammitglieder sieht der Client noch einfacher aus. Was ist ihre Aufgabe? Ordnung in die Positionen einer Sitzung bringen: Sobald eine Sitzung publiziert ist, können Teammitglieder sie öffnen, sehen die Positionen in ihrer Ordnung (beziehungsweise zu Anfang in der erfassten Reihenfolge) und ordnen sie um. Wenn sie mit ihrer Ordnung zufrieden sind, reichen sie sie ein zur Verrechnung mit allen anderen eingereichten, individuellen Ordnungen.

Hören Sie schon aus dieser Beschreibung die Interaktionen heraus? Oder wäre es Ihnen lieber, sich an sie schrittweise über Perspektiven und Dialoge heranzuarbeiten?

Ich denke, wir können einen Sprung wagen. Die Interaktionen für den Crowd-Client sind:

- Starten mit Öffnung einer Sitzung.
- Einreichen der eigenen Ordnung.

Wie die eigene Ordnung hergestellt wird, ist ein Detail der Benutzerschnittstelle. In einem GUI- oder Web-Frontend geschieht das anders als in einem Console-Frontend (Bild 7).



Theoretisch könnte ein solches verteiltes Ordnen auch ohne Benutzerschnittstelle ablaufen:

- Der PO legt in einem Dropbox-Verzeichnis eine Datei (*session.txt*) mit den zu ordnenden Positionen ab. In jeder Zeile steht eine Position mit einer ID, zum Beispiel einem Index 1..n.
- Die Teammitglieder öffnen diese Datei und speichern ihre individuelle Ordnung in einer neuen Datei (*peter.txt*, *paul.txt*, *maria.txt*) als Folge von Position-IDs, zum Beispiel „2, 1, 3“.
- Ein automatischer Prozess (oder ein von Hand gestarteter) erzeugt eine Ergebnisdatei, die die Positionen mit IDs und Bezeichnungen in einer weiteren Datei ablegt (*result.txt*).

Fertig. In dieser Variante gäbe es nur eine einzige Interaktion: den Start des Verrechnungsprozesses.

Was ich damit sagen will: Wir lassen uns oft durch Benutzerschnittstellen ins Bockshorn jagen. Und mit grafischen Benutzerschnittstellen, die hübsch und animiert und ganz CI-konform sein sollen, gleich doppelt. Die eigentlich interessante Verhaltensweise von Software hat oft nichts damit zu tun.

Darum finde ich es beim Slicing wichtig, die Benutzerschnittstelle so abstrakt wie möglich, so konkret wie nötig zu betrachten. Verlieren Sie nicht das Ziel aus den Augen: Es geht (zunächst) nur darum, die Entry Points zu finden. Denn sind die Entry Points definiert, kann die Arbeit am zentralen Verhalten der Software beginnen. Die Domäne liegt hinter der Schnittstelle der Processor-Integrationen von Sleepy Hollow.

Indem ich hier zwei Applikationen mit eigenen Frontends „herausanalysiert“ habe, bin ich also schon über einen Proof of Concept (PoC) weit hinausgegangen. Ob das Verfahren „relative Ordnung“ (oder konkreter: relative, vergleichende Schätzung) überhaupt interessant ist, könnte einfacher getestet werden.

Ein allererster PoC für die Machbarkeit könnte mit Google Sheet realisiert werden. Der PO trägt in einer Google-Sheet-Datei die Positionen ein, und die Teammitglieder „nummern“ diese in je eigenen Spalten nach ihren Vorstellun- ►

gen. Ständig wird eine gemeinschaftliche Ordnung auf einem zweiten Tabellenblatt angezeigt. Das sollte ein No-brainer sein.

So wäre die Herangehensweise effizient und effektiv. Ob es überhaupt mehr braucht als ein Google Sheet, würde sich herausstellen. Allerdings bliebe dabei der Programmierspaß auf der Strecke, und ich müsste mir ein anderes Szenario für die Demonstration von Slicing und Sleepy Hollow ausdenken.

Aber sehen wir einmal darüber hinweg. Wir sind hier in einem Sandkasten, in dem vieles nicht ganz real(istisch) ist. Zumindest einmal wollte ich jedoch so weit zurücktreten und reflektieren. Nicht, dass Slicing oder Sleepy Hollow in den Verruf kommen, Projektaufwände in die Höhe zu treiben. Nein, im Gegenteil: Es geht mir stets um „das rechte Maß“ (das oft kleiner ist als das, wohin ein Projektteam tendiert).

Die Entry Points für die Interaktionen

Für mein Gefühl ist genügend Klarheit an der Oberfläche hergestellt. Die grobe Struktur des Softwaresystems ist definiert, die Interaktionen sind herausgearbeitet. Damit bin ich bereit, unter die Oberfläche zu tauchen.

PO Frontend		Ressourcen		Signatur	
	Entry Point	Datenbank	Client ID	Request	Response
	start	read	read/write	sessionID:string	description:string, items:string[]
	publish	write		description:string, items:string[]	sessionID:string
	update	read			items:string[]
Crowd Frontend					
	start	read	read/write	sessionID:string	description:string, items:{index:number, title:string}[]
	submit	write		indexes:number[]	

Signaturen der Entry Points (Bild 8)

Unter der Oberfläche der Interaktionen, die in Dialogen von Workern in Applikationen stattfinden, liegt Code. Das Pendant von Interaktionen sind Entry Points, das heißt Funktionen, die im Rahmen der Interaktionen getriggert werden. Sie sind Ansatzpunkte für automatisierte Tests. Durch sie kann ein Softwaresystem „headless“ betrieben werden – oder eben auch mit verschiedenen Heads. Deshalb ist mir das Sleepy-Hollow-Architekturmuster so wichtig.

Bei geklärten Interaktionen ist also jetzt die Frage, wie dazu die Entry Points aussehen sollten. Das Command-Query-Separation-Prinzip (CQS) kann hier auch helfen, um noch weiter zu differenzieren und Entry Points nicht zu überladen.

Zu Entry Points gehört als Erstes eine Signatur mit typisierten Parametern und Rückgabewert. Für mich ist das in erster Linie ein „Wunschkonzert“: Wie stelle ich mir die Entry Points idealerweise vor? Wie vermeide ich, dass an ihnen (enge) Kopplung zwischen Head und Body entsteht?

Ich versuche es mit einem ersten Pass über die Interaktionen mit Blick durch die CQS-Brille:

PO-Frontend

- **Starten:** Eine Query, die gegebenenfalls eine Sitzung für eine gegebene ID herausucht.
- **Publizieren:** Ein Command, das eine Sitzung speichert, also den Datenbankzustand verändert.
- **Aktualisieren:** Eine Query, die individuelle Ordnungen liest und zu einer gemeinschaftlichen verrechnet (die nicht in der Datenbank gespeichert wird).

Crowd-Frontend

- **Starten:** Eine Query, die eine Sitzung und gegebenenfalls die schon vorhandene individuelle Ordnung eines Users herausucht.
- **Einreichen:** Ein Command, das eine individuelle Ordnung in der Datenbank speichert.

Aus CQS-Sicht sind die Interaktionen also sauber. Das ist ein guter Anfang. Jetzt weiter ins Detail: die Signaturen.

- Was soll an einen Entry Point übergeben werden? Was wurde in der Benutzerschnittstelle gesammelt?
- Was soll ein Entry Point liefern? Was soll in der Benutzerschnittstelle projiziert werden?
- Welche Seiteneffekte (Zustandsänderungen) treten auf? Welche Daten(strukturen) zum Beispiel in einer Datenbank sind wie beteiligt?

Bild 8 zeigt, wie ich mir die Signaturen und die groben Seiteneffekte vorstelle. Wie Sie sehen, ist aus der reinen Query *Starten* nun ein Hybrid geworden. Es wird nicht nur aus der Da-

```

1  type SessionID = string;
2  type Description = string;
3  type Item = string;
4
5  0 implementations
6  interface IPOProcessor {
7      // Start method
8      start(sessionID: SessionID): Promise<{
9          description: Description;
10         items: Item[];
11     }>;
12
13     // Publish method
14     publish(description:string, items:Item[]): Promise<{
15         sessionID: SessionID;
16     }>;
17
18     // Update method
19     update(): Promise<{
20         items: Item[];
21     }>;
22
23  0 implementations
24  interface ICrowdProcessor {
25      // Start method
26      start(sessionID: SessionID): Promise<{
27          description: Description;
28          items: Array<{
29              index: number;
30              title: string;
31          }>;
32     }>;
33
34     // Submit method
35     submit(indexes: number[]): Promise<void>;
36 }

```

Die Schnittstellen für die Entry Points (Bild 9)

tenbankressource und auch der ClientID-Ressource gelesen. Falls die Client-Instanz noch keine ID hat, wird auch eine geschrieben. Das halte ich aber für einen vernachlässigbaren Seiteneffekt, der auch nur einmal pro Instanz auftritt. Für mich bleibt das Starten also eine Query.

Eine KI wie ChatGPT oder Claude hilft mir, aus der Tabelle zwei Interfaces abzuleiten (Bild 9). Durch das Sleepy-Hollow-Architekturmuster weiß ich, dass und wo ich sie brauche (siehe unten).

Woher weiß ich, dass die Entry Points „gut und richtig“ sind? Ich weiß es nicht. Ich muss es auch nicht zu 100 Prozent wissen. Dieser erste aus der Anforderungsanalyse abgeleitete Entwurf für die Schnittstelle des Bodys muss nur gut genug sein, um mir Gedanken über Akzeptanztests machen zu können. Während ich die durchgehe, merke ich, ob die Entry Points wirklich plausibel sind. Test-first-Vorgehen hilft bei der Ausgestaltung von Schnittstellen. Eine Form von „eating your own dog food“.

Die Tests stellen sehr reduzierte Benutzerschnittstellen dar. Ich merke also schon mit ihnen, ob meine Idee der Zustandshaltung in den Prozessoren taugt. Haben Sie bemerkt, dass ich die Interfaces so gestaltet habe, dass ihre Implementationen zustandsbehaftet sein müssen? Sie sehen es zum Beispiel daran, dass bei `IPOProcessor.update()` keine `sessionID` übergeben wird. Woher weiß die Implementation, um welche Session es geht? Sie hat sich die Session vorher bei `start()` oder `publish()` gemerkt.

Prozessoren mit Zustand ziehe ich vor, weil damit mehr Logik testbar wird. Ein Head über dem Sleepy-Hollow-Body soll so wenig wie nur irgend möglich leisten. Es kann daher sogar dazu kommen, dass ich die Prozessoren nochmal mit einer weiteren Schicht umwickle, die dem UI Arbeit abnimmt und sie testbar macht. Mal sehen ...

Die Sleepy-Hollow-Architektur

Jetzt rede ich immer von Prozessoren, auf denen die Entry Points sitzen, aber wo sind die in der Architektur, wie ist deren Verhältnis zu den Strukturelementen, die ich schon herausgearbeitet habe?

Betrachten wir zur Erinnerung noch einmal das Sleepy-Hollow-Architekturmuster [3] in Bild 10. Zu sehen sind die Arbeitspferde einer Anwendung: Portale, Provider und Domäne. Darin steckt die verhaltenserzeugende Logik.

Ins Auge stechen jedoch mehr die gelben Integratoren. Ihre Aufgabe ist es, ganz ohne Logik die Arbeitspferde zu einem leistungsfähigen Gespann zusammenzuschirren. Diese Integratoren, die in einer Übersicht wie Bild 3 ganz bewusst nicht zu sehen sind, machen das Besondere der IODA-Architektur [6] und des darauf aufbauenden Sleepy-Hollow-Architekturmusters aus.

Als Body, der die eigentliche Leistung einer Anwendung erbringt, bezeichne ich

vor allem das, was von der Processor-Integration verbunden wird. Aber auch der Interactor kann noch dazugezählt werden, weil er ebenfalls dafür sorgt, dass Logik von den technologischen Feinheiten eines UI getrennt ist. Aber der Interactor ist optional. Der Body ist vergleichsweise leicht zu testen (auch wenn darin Ressourcenabhängigkeiten stecken).

Der Head ist das UI in der einen oder anderen Ausprägung. Es ist technologiespezifisch und schwer zu testen. Deshalb wird es vom Body getrennt, isoliert.

Wie dieses Muster auf das hiesige Softwaresystem angewendet werden kann, zeigt Bild 11. Die Strukturelemente der bisherigen „Zoomstufen“ finden sich darin – und auch die neuen Integrationsbausteine: App, Interactor und Processor.

Die Schnittstellen aus Bild 9 sind die für die Integratoren in der Mitte, die mit dem „You are here“-Symbol gekennzeichnet sind. Die Anforderungsanalyse hatte also vor allem das Ziel, genau diese Integrationen zu bestimmen und auszuprägen. Sie sind die Ansatzpunkte für Akzeptanztests; von ihnen aus werden die Schnittstellen der darunter hängenden schon sichtbaren und weiterer noch zu findender Bausteine getrieben.

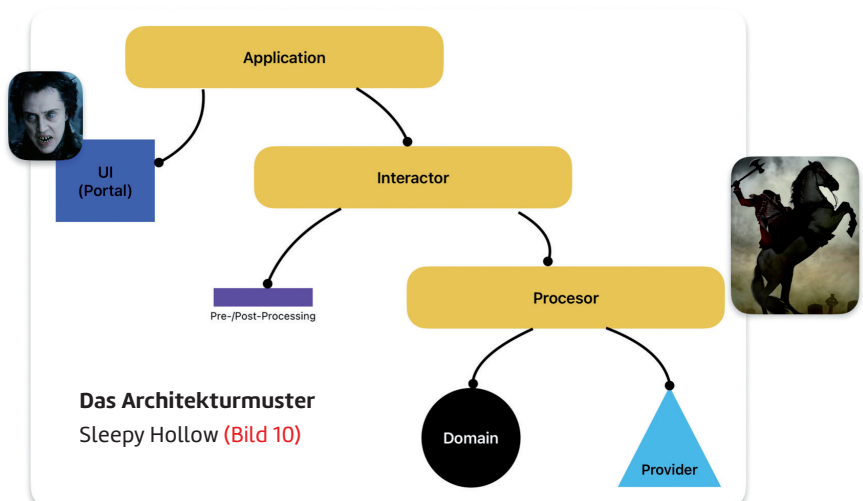
Die Verteilung auf Worker habe ich in Bild 11 bewusst übergangen. Sie trägt nichts zum Verständnis des grundlegenden Architekturmusters bei. Am Ende läuft der Durchstich je Interaktion mit oder ohne Verteilung vom Prozessor bis zum Provider mit Ressourcenzugriff.

Akzeptanztests für die Entry Points

Da stehe ich nun mit zwei Schnittstellen für Integrationen „irgendwo in der Mitte“ der Architektur. Was soll das? Ist das nicht ein bisschen wenig?

Nein, das ist der natürliche Endpunkt der Anforderungsanalyse. Der ist sogar schon viel detaillierter als alles, was üblicherweise bei der Anforderungsanalyse herauskommt. Überlegen Sie einmal, womit Sie Ihr PO sonst so zurücklässt, wenn er meint, die User Stories seien besprochen.

Mit dem Sleepy-Hollow-Architekturmuster ist von vornherein klar, dass es Prozessoren gibt, die „ein API“ mit Funktionen anbieten, die quasi eins zu eins den Interaktionen der Benutzerschnittstelle entsprechen. Sie wissen sogar, was ►



grundsätzlich darüber und darunter passiert. Genaue Kenntnis ist jedoch noch nicht nötig. Für den nächsten Schritt, den wahren Abschluss der Anforderungsanalyse, reicht es, dass Sie die Prozessoren mit ihren Schnittstellen kennen. Denn mit diesen Schnittstellen legen Sie den PO final auf den Grill: Sie fordern von ihm Testszenarien, die Sie dann mit den Entry Points umsetzen. Ohne solche Akzeptanztests sollten Sie den PO nicht ziehen lassen. Es droht Unterspezifikation mit bitteren Folgen: unnötige Kommunikation, Konflikte, Unzuverlässigkeit, Nachbesserungen.

Akzeptanztests, die für den PO Sinn ergeben, sind Teil des Full Kitting [5], das Sie betreiben sollten, bevor Sie sich zu weiterem Entwurf und Umsetzung zurückziehen. Sie sollten alles bei der Hand haben, was Sie brauchen, um gut passenden Code „in einem Rutsch“ liefern zu können. Das ist vor allem ... Klarheit. So viel Klarheit, wie Sie nur kriegen können. Klare Schnittstellen und klare Erwartungen an Output für Input sind das Mindeste.

Akzeptanztests sind natürlicherweise Integrationstests. Sie setzen an Bausteinen an, die auf die eine oder andere Weise nur „die Spitze des Eisbergs“ der Verarbeitung darstellen. Es stellt sich daher die Frage, ob Sie im Akzeptanztest hinunter bis auf die eigentlichen Ressourcen testen (hier: Dropbox-Dateien) oder ob Sie einfachere, billigere, besser kontrollierbare Ressourcen einschalten (Attrappen der einen oder anderen Art).

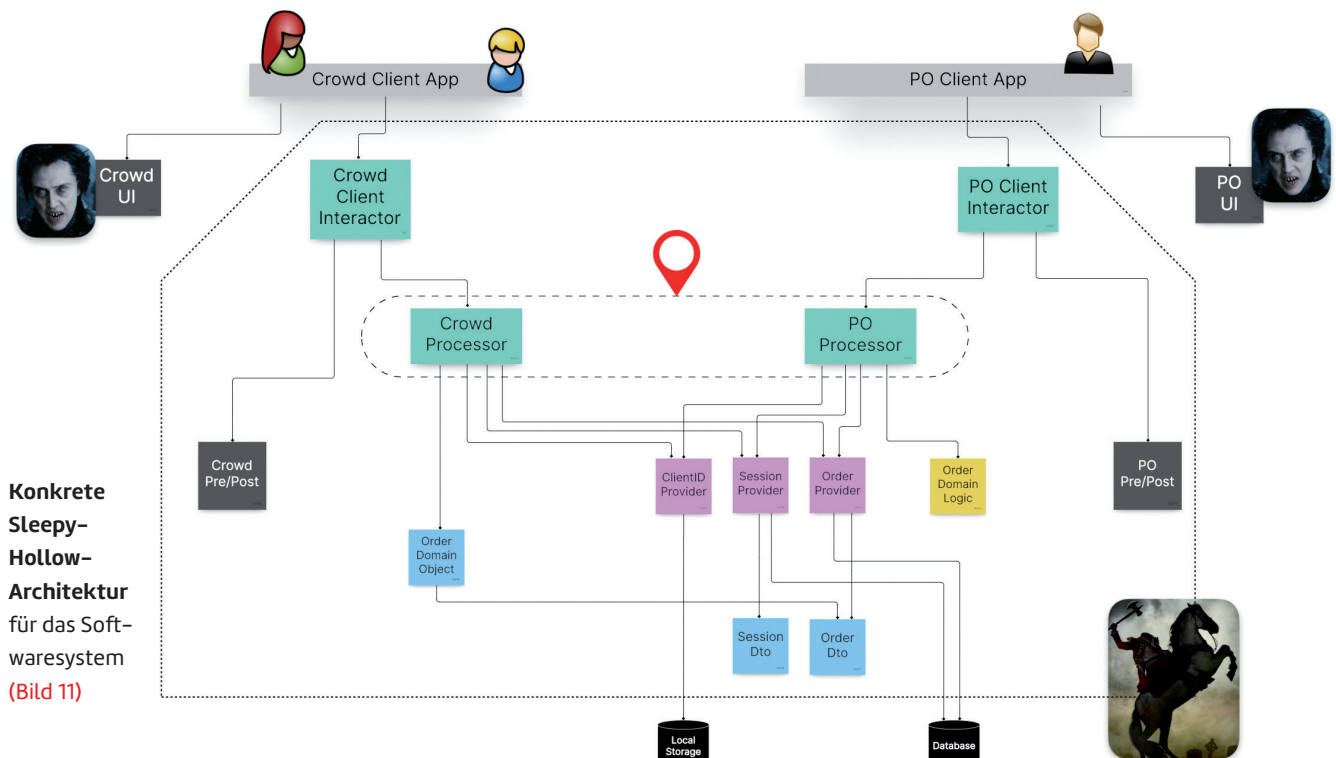
Diese Entscheidung möchte ich hier nicht weiter diskutieren. Nur so viel sei gesagt: Wenn Sie sich für Attrappen entscheiden, müssen Sie für Akzeptanztests auch die Schnittstellen der Provider für zu ersetzende Ressourcen kennen, um dafür Surrogate bauen und Prozessoren unterchieben zu können. Das erfordert zu diesem Zeitpunkt weiteren Entwurfsaufwand und verzögert das Aufsetzen von Akzeptanztests.

Die Formulierung der Testszenarien selbst ist davon jedoch unabhängig. Darin spielen nur die unmittelbar zu testenden Bausteine (hier: die Prozessoren) eine Rolle.

Aufgabe von Akzeptanztests ist nicht, die komplette Logik eines Bodys durchzuexerzieren. Es geht um das Big Picture, die wichtigsten Zusammenhänge, ein Gefühl dafür, ob typische Use Cases abgedeckt sind.

Für das vergleichende Ordnen mögen dies repräsentative Akzeptanztests sein:

- Neue Sitzung publizieren.
 - Starten ohne SessionID.
 - Publizieren einer Session mit der Beschreibung „B“ und den Einträgen „X“, „Y“, „Z“. Es wird eine SessionID geliefert.
 - Aktualisieren der Session in der Erwartung, dass die Einträge „X“, „Y“, „Z“ in der Reihenfolge geliefert werden.
- Sitzung individuell das erste Mal ordnen.
 - Starten mit einer SessionID. Es werden Beschreibung „B“ und die Einträge [{0, „X“}, {1, „Y“}, {2, „Z“}] geliefert.
 - Einreichen der individuellen Ordnung [2,1,0].
- Sitzung individuell das erste Mal durch einen weiteren Client ordnen lassen.
 - Starten mit einer SessionID. Es werden Beschreibung „B“ und die Einträge [{0, „X“}, {1, „Y“}, {2, „Z“}] geliefert.
 - Einreichen der individuellen Ordnung [0, 2, 1].
- Starten einer existierenden Sitzung und Aktualisieren nach individuellen Ordnungen.
 - Starten mit SessionID; es werden Beschreibung „B“ und die Einträge „X“, „Y“, „Z“ geliefert.



- Aktualisieren der Session. Die Erwartung ist, dass sich die Ordnung gegenüber der ursprünglichen verändert hat. Sie soll nun „Z“, „X“, „Y“ lauten.

Ich habe diese Tests zwar separat formuliert, doch für mich spannen sie eigentlich einen Bogen. Ich würde sie zusammen laufen lassen. Das nenne ich ein Szenario. In diesem Szenario wird eine Sitzung definiert und publiziert, von zwei Teammitgliedern individuell geordnet und am Ende das Ergebnis betrachtet. Da ist alles Wesentliche dabei, das in der Praxis über zwei Clients verteilt passiert, nur ist es hier für besseren Überblick und bessere Testbarkeit in einem Test zusammengefasst.

Auf diese Weise muss ich mir weniger Gedanken über die Überprüfung von Zuständen machen. Wird die Einreichung einer individuellen Ordnung wirklich gespeichert? Das merke ich am Ende, wenn das Resultat stimmt oder nicht.

Allerdings fühle ich jetzt schon, dass hier und da die Prozessor-Schnittstellen nicht ausreichen. Wie unterscheide ich zum Beispiel die beiden Crowd-Clients? Sie laufen während des Tests ja im selben Prozess, dürfen jedoch nicht dieselbe ClientID haben. Brauche ich dafür eine Attrappe für den ClientID-Provider? Das ist ein Detail, das ich bei der Implementation des Tests entscheide.

Wie steht es mit der Abdeckung? Nicht der Code-Pfadabdeckung, sondern der Feature-Abdeckung. Ist in dem Akzeptanztest alles Interessante getestet? Nein. Es fehlt die Veränderung einer Sitzung, wie in **Bild 5** als Möglichkeit gezeigt. Das wäre einen zweiten Szenariotest wert:

- Neue Sitzung publizieren wie oben.
- Sitzung individuell ordnen wie oben.
- Existierende Sitzung verändern.
 - Starten der gerade publizierten Sitzung.
 - Publizieren mit den Einträgen „X“, „Y“, „Z“, „AA“; es wird eine neue Sitzungsnummer zurückgeliefert.
- Sitzung individuell ordnen.
 - Starten mit der neuen Sitzungsnummer; es sollten die Beschreibung „B“ und die Einträge [{2, „Z“}, {1, „Y“}, {0, „X“}, {3, „AA“}] zurückgeliefert werden.
 - Einreichen der individuellen Ordnung [2, 3, 0, 1].
- Starten der neuen Sitzung und Aktualisieren.
 - Starten mit neuer SessionID; es sollte „B“ und [„X“, „Y“, „Z“, „AA“] geliefert werden.
 - Aktualisieren; das Ergebnis sollte [„Z“, „AA“, „X“, „Y“] sein, weil nur ein Teammitglied eine Ordnung eingereicht hat.

Das sind für einen PO greifbare Formulierungen, würde ich sagen. Sie sind frei von technischen Feinheiten, und trotzdem sind sie präzise für mich als Entwickler. Ich weiß genau, welche Funktion ich wie aufzurufen habe – während der PO nur in Interaktionen denkt. So begegnen wir uns auf Augenhöhe:

- Der PO bekommt Klarheit darüber, was er will, was seine Erwartungen sind. Es ist zu wenig, wenn er nur mit den Händen wedelt und sagt: „Na ja, du weißt schon ...“ oder „Du bist doch der Experte“. Klar, ich bin der Experte, Sie sind der Experte. Für Codierung von Transformationen, die

in Anforderungen versteckt sind. Wir sind keine Experten für Anforderungen. Die muss der PO schon selbst definieren. Präzise!

- Ich bekomme Klarheit darüber, wie meine grobe Architektur aussieht und wo die Startpunkte liegen, mit denen ich in Entwurf und Codierung einsteige.

Zwischenstand

Ausgehend von einer Anforderungsskizze habe ich mit Slicing eine Liste von Funktionen bestimmt, mit denen sich die Anforderungen verhaltensmäßig abdecken lassen. Für zwei Verhaltensszenarien habe ich konkrete Vorstellungen entwickelt, die ich als Akzeptanztests gegen die Funktionen laufen lassen kann. Damit habe ich eine Ziellinie gezogen. Die ist erreicht, sobald diese Tests grün sind.

Das Sleepy-Hollow-Architekturmuster hat mir ein Terrain geliefert, in dem ich genau verortet bin. Ich weiß, wo ich stehe, was ich habe – und ich weiß, was noch fehlt, was nun näher zu entwerfen und dann zu codieren ist.

Jetzt habe ich alles beieinander, um mit möglichst wenig Kommunikation mit einem PO meinen eigentlichen Job zu machen. Ich habe Full Kitting so weit wie möglich betrieben. Sicher gibt es noch eine Grauzone, nicht alles lässt sich vor der Codierung haarklein bestimmen. Ich will ja auch nicht in einen Wasserfall zurückfallen. Doch ich empfinde, dass ich mich zumindest gut präpariert habe. Ohne Vorbereitung in das Abenteuer Codierung einzusteigen ist zu wenig.

Jetzt ist mein Rucksack gepackt, und ich habe eine Karte in der Hand. Damit geht es nächstes Mal los in die Implementation. ■

- [1] Ralf Westphal, *Strukturiert zerlegen*, dotnetpro 4/2024, Seite 42 ff., www.dotnetpro.de/A2404Slicing
- [2] Ralf Westphal, *Inkrementale innerhalb von Interaktionen finden*, dotnetpro 9/2024, Seite 32 ff., www.dotnetpro.de/A2409Slicing
- [3] Ralf Westphal, *Den Kopf vom Körper trennen*, dotnetpro 11/2024, Seite 46 ff., www.dotnetpro.de/A2411SleepyHollow
- [4] Ralf Westphal, *TotalOrder: Konsequent vergleichend schätzen*, www.dotnetpro.de/SL2412SleepyHollow1
- [5] Goldratt UK, *A Simple but Highly Effective Way to Increase Output: Full-Kitting*, www.dotnetpro.de/SL2412SleepyHollow2
- [6] Ralf Westphal, *IODA Architecture*, www.dotnetpro.de/SL2412SleepyHollow3



Ralf Westphal

ist Trainer, Berater und Mitgründer der Clean Code Developer Initiative (<https://clean-code-developer.de>). Seine Schwerpunkte sind dauerhaft hohe Produktivität für die Softwareentwicklung und zukunftsfähige Teamorganisation.

<https://ralfw.de>

dnpCode

A2412SleepyHollow